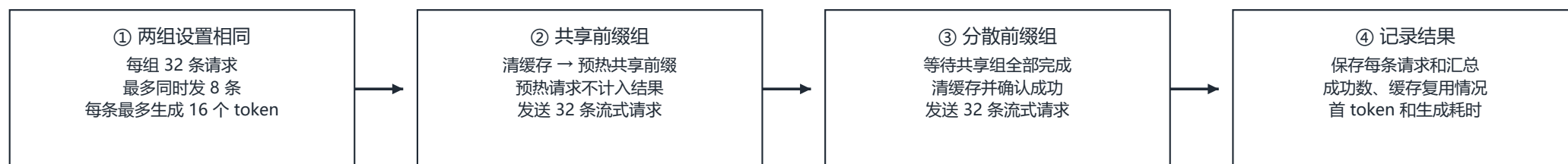


要比较什么

比较两组请求：一组有相同的开头，另一组的开头各不相同。看看 SGLang 能不能重复使用已经缓存的 KV，以及这样做能不能减少这轮需要重新处理的输入 token。

实验怎么做



这里几个词的意思

Prefill：模型处理用户输入的阶段，会为输入算出 KV。开头和以前的请求相同时，缓存里的那部分 KV 可以直接用；只有没缓存的部分需要重新算。

Decode：模型根据输入一步一步生成回答。流式返回是把已经生成的回答分段发给客户端；它和 Decode 有关，但不是同一件事。

缓存复用比例：这次重复使用的输入 token 数，占全部输入 token 数的比例。

TTFT：等到第一个输出 token 的时间；TPOT：后续输出 token 之间平均隔多久；E2E：一条请求从开始到结束的总时间。p50 是中间位置的时间，p95 表示 100 条里约 95 条不超过这个时间。

为什么相同开头可以复用 KV

模型按顺序处理输入时，前面的内容不会因为后面接了不同的文字而改变。只要模型和前面那段 token 的顺序相同，前面已经算出的 KV 就能重复使用；不同的后半段仍要重新处理。

两轮使用同样的请求和设置。两轮的缓存复用数量相同，但耗时和每秒完成请求数不完全一样。

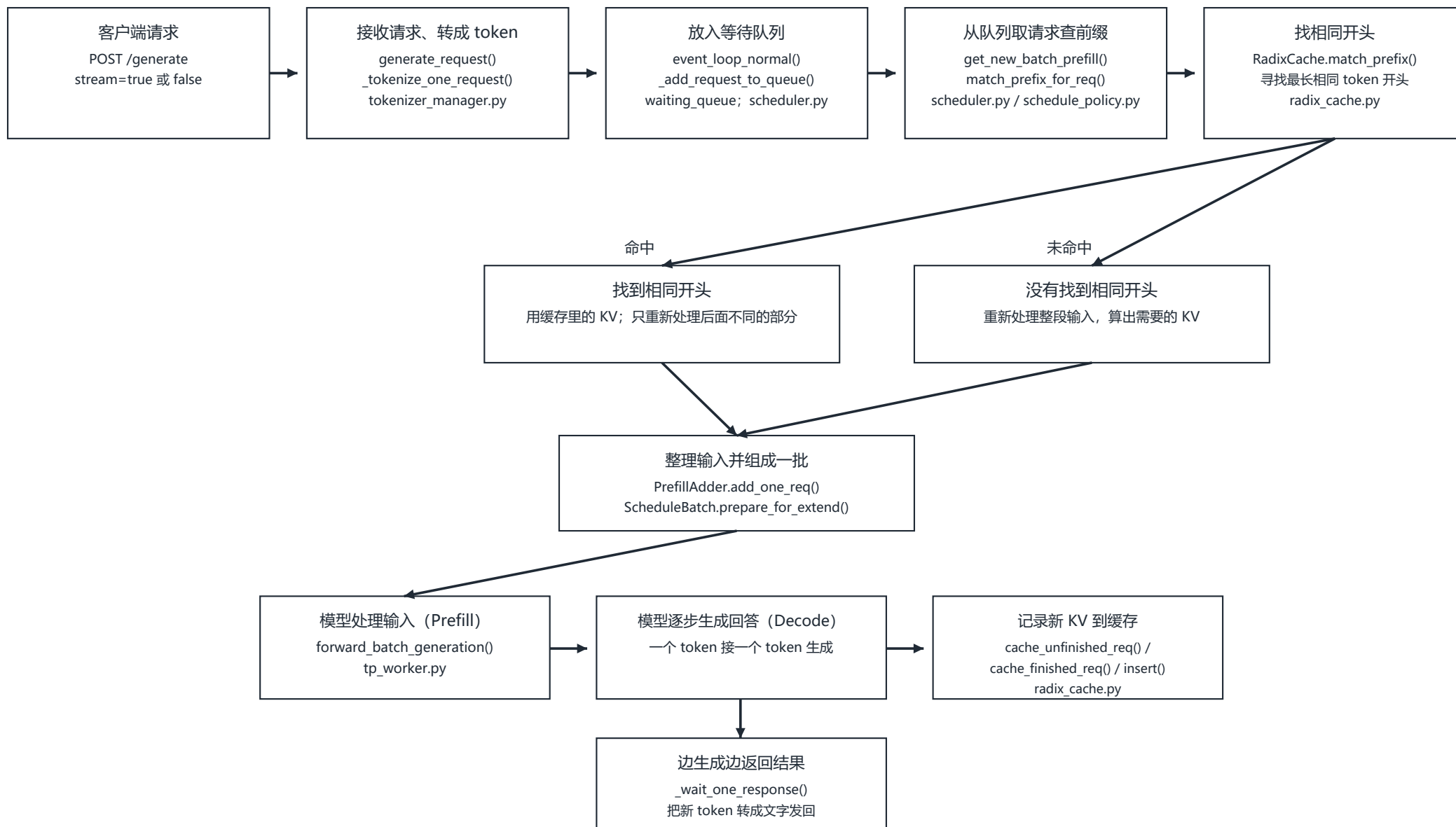
轮次	请求组	成功数	复用比例	复用 token	重新处理的输入 token	每秒请求数
run-1	共享前缀	32/32	18.45%	16,384	72,428	7.89
run-1	分散前缀	32/32	0.00%	0	88,812	6.78
run-2	共享前缀	32/32	18.45%	16,384	72,428	4.38
run-2	分散前缀	32/32	0.00%	0	88,812	6.06

表 1. 复用比例 = 复用的 token 数 ÷ 全部输入 token 数；“重新处理的输入 token” = 全部输入 token - 复用 token。每秒请求数按成功请求数和该组耗时计算。

轮次	请求组	首 token 等待 p50/p95 (s)	后续 token 间隔 p50/p95 (s)	整条请求用时 p50/p95 (s)
run-1	共享前缀	0.481 / 0.871	0.034 / 0.060	1.002 / 1.113
run-1	分散前缀	0.554 / 1.027	0.039 / 0.068	1.176 / 1.279
run-2	共享前缀	0.992 / 2.730	0.042 / 0.082	1.422 / 3.000
run-2	分散前缀	0.664 / 1.241	0.039 / 0.081	1.297 / 1.457

结论

- 两轮里，共享开头的一组都重复用了 16,384 个 token；需要重新处理的输入 token 从 88,812 个降到 72,428 个，少了约 18.45%。两组请求都全部成功。
- 第一次测试时，共享组的等待时间较短；第二次测试时，共享组反而更慢。说明这两轮能看出缓存少算了一部分输入，但不能说明每次都一定更快。
- 耗时还会受到电脑当时的负载和请求排队情况影响。生成回答的速度也没有在两轮中稳定变快。



用户把请求发到 /generate 后，TokenizerManager.generate_request 接收请求。如果传来的是文字，_tokenize_one_request 会把文字转成 token IDs；如果请求里已经有 input_ids，就直接使用。接下来 Scheduler 把请求放进 waiting_queue（等待队列），等着安排运行。

Scheduler 从等待队列取出请求后，会用 RadixCache.match_prefix 查它的 token 开头是否和缓存中的内容相同。相同的部分可以用以前算好的 KV；后面不同的输入还要重新处理。若没有相同开头，整段输入都要处理。PrefillAdder 和 ScheduleBatch 会把本轮要处理的请求整理成一批交给模型。

模型先做 Prefill：处理这次没有缓存的输入，并算出对应的 KV。然后做 Decode：参考输入和 KV，一个 token 接一个 token 地生成回答。新算出的 KV 会放进 KV 缓存，并记下它对应的 token 开头，供之后的请求查找。它不是把旧 KV 和新 KV 粘成一块。

开启 stream 后，TokenizerManager 会陆续收到新生成的 token，把它们转成文字发给客户端。这样用户可以边等边看到回答；模型还会继续生成，直到回答完成。

关键函数与官方源码位置

接收、文字转 token、发回结果：generate_request / _tokenize_one_request / _wait_one_response (managers/tokenizer_manager.py)

等待队列、安排请求运行：event_loop_normal / get_new_batch_prefill (managers/scheduler.py)

查找前缀、整理请求：match_prefix_for_req / PrefillAdder.add_one_req (managers/schedule_policy.py)

查找和保存 KV：RadixCache.match_prefix / cache_finished_req / cache_unfinished_req / insert (mem_cache/radix_cache.py)

准备模型输入：Req.init_next_round_input / ScheduleBatch.prepare_for_extend (managers/schedule_batch.py)

模型运行：TpModelWorker.forward_batch_generation (managers/tp_worker.py)

简单区分：waiting_queue 是等着运行的请求队伍；Prefill 是处理输入，Decode 是生成回答；ScheduleBatch 是整理好后交给模型的一批请求。这些是队列或步骤名称，不是函数。